Programación por bloques

Nombre: _____ Fecha: _____

Qué vas a aprender

- Explicar qué es un programa y un lenguaje de programación
- Reconocer las ventajas de los lenguajes por bloques y algunos entornos
- Identificar los tipos de bloques: eventos, acciones, control, sensores y datos
- Crear, probar y depurar un programa sencillo por bloques

Para pensar

Si a un robot le dices «tráeme un vaso de agua», no se mueve. ¿Qué le falta para hacerlo?

Explicación

¿Qué es programar?

Un programa es una lista de **instrucciones** ordenadas que le dice a un ordenador o a otra máquina lo que tiene que hacer. Todas las aplicaciones del móvil, los videojuegos, las páginas web, el control de un semáforo o el de una lavadora son programas. Programar es escribir esas instrucciones. Como los ordenadores no entienden nuestro idioma, las instrucciones se escriben en un lenguaje de programación, un lenguaje con unas palabras y unas reglas muy precisas, que luego se traduce a ceros y unos. Hay muchos: Python, JavaScript, C, Java... El texto escrito en ellos se llama **código**. Para programar bien no basta con conocer el lenguaje: hay que saber **pensar el problema**, dividirlo en partes más sencillas y ordenar los pasos, algo que se llama pensamiento computacional. Programar no es solo cosa de informáticos: hoy se usa en ciencia, arte, música, medicina, ingeniería y casi cualquier profesión.

undefined

Los lenguajes por bloques

Escribir código en un lenguaje de texto es difícil al principio: un punto y coma olvidado o una palabra mal escrita hacen que el programa no funcione. Para aprender, se crearon los lenguajes de programación por bloques, en los que las instrucciones son **piezas de colores** que se arrastran con el ratón y se encajan como un puzle. Tienen muchas ventajas: no hay errores de escritura, porque las instrucciones ya están escritas; los bloques solo encajan donde tiene sentido; los colores indican la categoría de cada bloque; y se ve el resultado al momento. Así uno se puede concentrar en la **lógica** del programa. Los entornos más conocidos son Scratch, para crear historias, animaciones y juegos; MakeCode, para programar placas como la **micro:bit**; **App Inventor**, para crear aplicaciones de móvil; y los editores de robots educativos como mBlock. Lo que se aprende con bloques (secuencias, bucles, condiciones, variables) sirve después para cualquier lenguaje de texto.



las piezas solo encajan donde tienen sentido

Tipos de bloques

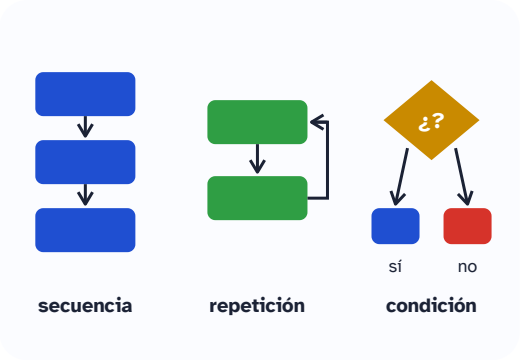
En todos los entornos de bloques, las instrucciones se agrupan en **categorías**, cada una con un color. Los bloques de eventos inician el programa cuando ocurre algo: «al hacer clic en la bandera verde», «al pulsar una tecla», «al iniciar», «al agitar». Tienen forma de sombrero porque siempre van arriba. Los bloques de acción hacen algo visible: mover, girar, mostrar un dibujo, encender una luz, tocar un sonido, decir un mensaje. Los bloques de control deciden el orden en que se ejecutan las instrucciones: esperar, repetir (bucles) o hacer algo solo si se cumple una condición («si… entonces»). Los bloques sensores obtienen información del entorno: si se toca algo, si se pulsa una tecla, la temperatura, la luz, la posición del ratón. Los bloques de **operadores** hacen cálculos y comparaciones (sumar, mayor que, número al azar). Y los bloques de **datos** o **variables** guardan valores que pueden cambiar, como los puntos de un juego o el tiempo que queda.

Categoría	Para qué sirve	Ejemplo
Eventos	empezar cuando pasa algo	al pulsar el botón A
Acción	hacer algo visible o audible	mostrar un icono, mover 10 pasos
Control	esperar, repetir, decidir	repetir 4 veces, si… entonces
Sensores	leer el entorno	temperatura, ¿tocando borde?
Operadores	calcular y comparar	número al azar, mayor que
Variables	guardar valores que cambian	puntos, vidas

undefined

Secuencia, repetición y condición

Casi todos los programas se construyen combinando tres estructuras. La secuencia: las instrucciones se ejecutan **una detrás de otra**, en el orden en que están colocadas, de arriba abajo. El orden importa: no es lo mismo «girar y avanzar» que «avanzar y girar». La repetición o **bucle**: un grupo de instrucciones se repite un número de veces («repetir 4 veces») o para siempre («por siempre»). Así se evita escribir lo mismo muchas veces: para dibujar un cuadrado basta con repetir 4 veces «avanzar y girar 90°». La condición: un grupo de instrucciones solo se ejecuta **si se cumple algo** («si la temperatura es mayor que 25, mostrar un sol»), y se puede añadir qué hacer en caso contrario («si no, mostrar una nube»). Combinando estas tres estructuras con eventos, sensores y variables se pueden crear programas sorprendentemente complejos, desde un termómetro digital hasta un videojuego.



Un ejemplo con la placa micro:bit

La micro:bit es una pequeña placa programable del tamaño de una tarjeta, con una matriz de 25 luces LED, dos botones, sensores de luz, temperatura y movimiento, brújula y radio. Se programa gratis desde el navegador con **MakeCode**, que incluye un simulador para probar los programas sin tener la placa. Veamos un **dado electrónico**. Necesitamos: un evento, «al agitar»; un bloque de variables, «fijar *tirada* a número al azar entre 1 y 6»; y un bloque de acción, «mostrar número *tirada*». Al agitar la placa, el sensor de movimiento lo detecta, se elige un número al azar, se guarda en la variable y aparece en las luces. Podemos mejorarlo: con un bloque «si *tirada* = 6 entonces» mostrar además una carita feliz, o con un bucle «repetir 5 veces» mostrar números al azar muy rápido antes del resultado final, para que parezca que el dado rueda. Este pequeño proyecto ya usa eventos, variables, operadores, condiciones y bucles.

undefined

Probar y depurar

Es normalísimo que un programa no funcione bien a la primera: hasta los programadores profesionales cometen errores constantemente. A los errores de un programa se les llama bugs («bichos» en inglés), y buscarlos y corregirlos es depurar. Los errores más frecuentes al programar con bloques son: colocar los bloques en un **orden equivocado**; olvidar el **bloque de evento**, de modo que el programa nunca empieza; poner un número incorrecto (girar 9 grados en vez de 90); usar «repetir» en lugar de «por siempre», o al revés; y no **reiniciar** las variables o la posición al empezar. Para depurar conviene: ejecutar el programa **paso a paso**, observando qué hace cada bloque; probar **una parte cada vez**; leer el programa en voz alta, como si fuéramos la máquina; añadir bloques que muestren el valor de las variables; y cambiar **una sola cosa** cada vez para ver qué efecto tiene. Probar con distintos casos, también los raros, ayuda a encontrar errores escondidos.

undefined

Cómo crear un proyecto de programación

Un proyecto de programación sigue los mismos pasos que el proceso tecnológico. 1) **Definir** qué queremos que haga el programa, con condiciones concretas: «un juego en el que un gato persigue al ratón y gana puntos cada vez que lo toca». 2) **Diseñar**: dividir el problema en partes (los personajes, el movimiento, los puntos, el final) y describir los pasos de cada una, con palabras o con un diagrama, antes de tocar el ordenador. 3) **Programar** cada parte, empezando por la más sencilla. 4) **Probar y depurar** cada parte antes de pasar a la siguiente. 5) **Mejorar**: añadir niveles, sonidos, efectos. 6) **Compartir**: en muchos entornos, como Scratch, se pueden publicar los proyectos para que otros los usen y los «remezclen». Al compartir hay que respetar a los demás, citar si se ha partido del proyecto de otra persona y no usar imágenes o música sin permiso. También es importante **guardar a menudo** y con nombres claros, para no perder el trabajo.

undefined

La programación por bloques en ocho ideas



Ejercicios

1. **¿Qué es un programa?**

2. **Escribe tres ventajas de los lenguajes por bloques.**

3. **Nombra tres entornos de programación por bloques.**

4. **¿Para qué sirven los bloques de eventos?**

5. **¿Qué es un bucle? Pon un ejemplo.**

6. **¿Qué es una condición?**

7. **¿Qué es una variable? Pon un ejemplo.**

8. **¿Qué es depurar?**

9. **Escribe tres consejos para depurar.**

10. **Escribe los pasos de un proyecto de programación.**

Practica

1. **Los bloques que inician un programa son de...**
   - **A)** eventos
   - **B)** operadores
   - **C)** apariencia
2. **La placa que se programa con MakeCode es la...**
   - **A)** micro:bit
   - **B)** placa base
   - **C)** tarjeta gráfica
3. **Un bucle sirve para...**
   - **A)** repetir
   - **B)** decidir
   - **C)** guardar
4. **«Si... entonces» es una...**
   - **A)** condición
   - **B)** variable
   - **C)** secuencia
5. **Los puntos de un juego se guardan en una...**
   - **A)** variable
   - **B)** condición
   - **C)** secuencia
6. **Un error en un programa se llama...**
   - **A)** bug
   - **B)** bit
   - **C)** byte
7. **Para un cuadrado se repite 4 veces girar...**
   - **A)** 90°
   - **B)** 45°
   - **C)** 120°
8. **En una secuencia las instrucciones van...**
   - **A)** una tras otra

B) todas a la vez

C) al azar

9. **Los bloques sensores sirven para...**

A) leer el entorno

B) repetir

C) dibujar

10. **Scratch sirve sobre todo para crear...**

A) juegos y animaciones

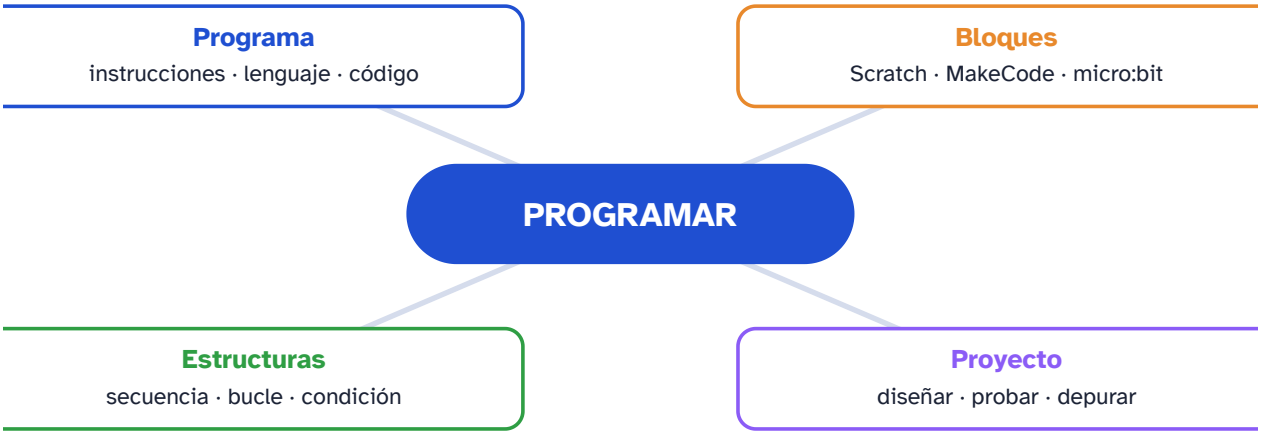
B) hojas de cálculo

C) sistemas operativos

Explícalo con tus palabras

Incluye: Qué es un programa y por qué una máquina necesita instrucciones precisas; Qué son la secuencia, el bucle y la condición; Cómo se busca un error en un programa.

Resumen



Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Programa

Lista ordenada de instrucciones para una máquina.

Lenguaje de programación

Lenguaje con reglas precisas para escribir programas.

Evento

Algo que ocurre e inicia un programa.

Secuencia

Instrucciones que se ejecutan una tras otra.

Bucle

Repetición de un grupo de instrucciones.

Condición

Instrucciones que solo se ejecutan si se cumple algo.

Variable

Espacio que guarda un valor que cambia.

Depurar

Buscar y corregir errores.

Soluciones

1. **A)** Eventos.
2. **A)** micro:bit.
3. **A)** Repetir.
4. **A)** Condición.
5. **A)** Variable.
6. **A)** Bug.
7. **A)** 90°.
8. **A)** En orden.
9. **A)** Leer datos.
10. **A)** Juegos, historias, animaciones.

Soluciones de los ejercicios

1. Una lista de instrucciones ordenadas que dice a una máquina lo que tiene que hacer.
2. No hay errores de escritura, los bloques solo encajan donde tienen sentido y se ve el resultado al momento.
3. Scratch, MakeCode y App Inventor.
4. Para iniciar el programa cuando ocurre algo.
5. Una repetición de instrucciones: repetir 4 veces avanzar y girar 90° para hacer un cuadrado.
6. Una estructura que ejecuta instrucciones solo si se cumple algo: si… entonces.
7. Un espacio que guarda un valor que puede cambiar, como los puntos de un juego.
8. Buscar y corregir los errores de un programa.
9. Ejecutar paso a paso, probar una parte cada vez y cambiar una sola cosa cada vez.
10. Definir, diseñar, programar, probar y depurar, mejorar y compartir.